



Project Report: Open Source Modular Controller Spring 2026

Prepared by:

[Ethan Dwyer](#) [Devon Flanery](#) [Gabriel Fosdick](#) [Brady Johnston](#) [Andres Salinas](#) [Stephen Venerable](#)



Table of Contents:

Table of Contents:.....	2
Executive Summary:.....	3
Core Technology.....	3
Innovation.....	3
Development Highlights.....	3
Impact.....	3
Main Product Image:.....	4
Project Description:.....	4
Methodology:.....	5
System Architecture.....	5
USB HID Implementation.....	5
Component Selection and Design Considerations.....	6
Mechanical and Ergonomic Design.....	6
Summary.....	7
Bill of Materials:.....	8
Tools Used:.....	10
Software.....	10
Hardware.....	11
Assembly.....	12
Characterization Data:.....	19
Input Testing.....	19
Ergonomic Design.....	19
Hardware Continuity.....	20
Standards:.....	22
Recommendations for Future Work:.....	22
Sponsors/Acknowledgements:.....	23
References:.....	23
Appendix:.....	24

Executive Summary:

The goal of this project is to design and develop an open-source modular game controller compatible with modern gaming and computing systems. The controller features interchangeable input modules that enable users to customize both the layout and physical placement of controls based on game genre, personal preference, or accessibility needs. By supporting a variety of input types and configurations, the system can be tailored for diverse users and applications, from video games to robotics control. Once assembled, the controller connects directly to standard devices using USB and is recognized as a Human Interface Device (HID) without additional drivers.

Core Technology

At the foundation of the system is the TinyUSB library, which allows communication within the USB HID framework. This approach eliminates the need for dedicated drivers and ensures compatibility across Windows, macOS, and Linux platforms, providing a plug-and-play experience for end users. On top of that is the i2c protocol that communicates between the modules and the Microcontroller, that handles the identification and input data streaming.

Innovation

The project's defining innovation lies in its fully modular and spatially flexible architecture. Each controller consists of removable button and joystick modules that attach to detachable receiver boards. These receiver boards can be placed anywhere on a custom chassis, giving users the freedom to design their own controller shapes and ergonomics. This allows complete reconfiguration of both the electronic and physical layout, something not possible with commercial controllers. The system is designed to be approachable for those with little to no electronics experience while remaining highly extensible for makers and developers.

Development Highlights

Over the course of the semester, the team refined and developed the second-generation receiver boards and the two modules: a button input and a joystick, with the addition of a bumper/trigger module. The base system successfully interfaced with a Windows PC and functioned in real-world gaming scenarios. Future work will expand on this foundation by adding module identification and refining the communication protocol to support more features.

Impact

This project represents a meaningful advancement in customizable controller design. Its modular and repositionable hardware offers new possibilities for makers, competitive gamers, and individuals with physical disabilities, enabling truly personalized ergonomics and input arrangements. By combining open-source firmware with hardware flexibility, the system empowers users to create, share, and refine their own control interfaces, making high-performance and adaptive hardware accessible to everyone.

Main Product Image:



Figure 1: Main Controller Image

Project Description:

The project was the design and implementation of a functional prototype for a modular, open-source game controller compatible with modern computing systems. The primary goal was to develop a controller system in which every major input—joysticks, buttons, and future modules—can be physically rearranged or replaced, giving users the freedom to build a controller layout that matches their play style or accessibility needs.

The first semester's development began with a breadboard-based proof of concept used to verify signal routing, input functionality, and USB communication. Once validated, the team progressed to designing custom printed circuit boards (PCBs) for the joystick and button modules, the receiver boards, and an ESP32-based carrier board serving as the main controller.

This second semester's focused on the modularity and hot-swapability of the different modules and making the enclosure more comfortable for the users . This was done by switching the modules to integrated circuits that have i2c compatibility so all the modules can share the same communication bus.

A 3D-printed enclosure was then modeled and fabricated to house the assembled electronics. Each receiver board connects to the module boards through pogo header pins or JST connectors, allowing the modules to be placed anywhere within a custom chassis. This spatial flexibility is the defining feature of the system, enabling users to create their own controller shapes and layouts to suit personal or accessibility requirements.

The controller's firmware utilizes the TinyUSB library to manage communication through the USB Human Interface Device (HID) framework, ensuring plug-and-play functionality without custom drivers. The current prototype successfully interfaces with Windows systems and is fully recognized as a standard game controller. Although additional inputs such as bumpers and triggers are planned for future iterations, the completed prototype establishes a robust hardware and software foundation for a fully modular and reconfigurable controller platform.

Methodology:

System Architecture

The modular controller system was built around the ESP32-S3 microcontroller, chosen for its native support of both USB and Bluetooth HID (Human Interface Device) communication. This capability allows the controller to function as a USB peripheral without the need for custom drivers and provide wireless compatibility for future iterations of this project. The ESP32-S3 communicates the controller's given inputs from each module to the host system. The transmission between the modules and microcontroller is done through a main receiver board.

The main receive board hosts the controller with standard connector pins and the modules with either pogo pins or JST connectors depending on the controller shell design. Using I2C the controller communicates with each module based on its type and position. Each module has a different I2C address based on which type of module it is (joystick, button, etc.) and an address translator placed at each slot provides a different position to the microcontroller.

USB HID Implementation

Communication between the controller and host system relies on the USB HID standard, which defines how input devices transmit data to computers and consoles. When a device is connected, the host begins a process called enumeration, during which it requests a series of descriptors from the device. These descriptors identify the device type, power requirements, and data structure:

- Device Descriptor: Specifies the vendor ID, product ID, and device class.
- Configuration and Interface Descriptors: Define endpoints, interfaces, and required power.

- HID and Report Descriptors: Describe the logical format of inputs such as buttons and joystick axes.
- String Descriptors (optional): Provide human-readable information, such as product name or manufacturer.

Once the enumeration sequence is complete, the device begins sending HID reports, which contain button states and analog axis values. These reports are continuously exchanged with the host to provide real-time input recognition. Using the TinyUSB library, the ESP32-S3 handles all enumeration, descriptor generation, and report formatting internally, allowing the controller to be recognized as a standard gamepad on Windows without additional drivers.

Component Selection and Design Considerations

Component selection emphasized functionality, modularity, and cost efficiency. The ESP32-S3 was selected for its integrated USB OTG interface, Bluetooth LE support, and sufficient processing resources for handling multiple modular inputs. Standard pin headers were chosen as the initial connection method between modules and receiver boards due to their availability, affordability, and mechanical stability. The current version transitions to magnetic pogo-pin connectors to simplify module swapping and improve durability.

All printed circuit boards (PCBs) were designed in-house and fabricated through a low-cost prototyping service. Careful attention was given to connector orientation, trace routing, and power distribution to ensure reliability and ease of assembly.

I2C Components

To further increase the modularity and functionality of the controller, we wanted to be able to automatically detect module changes while the controller was in use. We decided to implement this using I2C. Using SPI or adding small microcontrollers to each module were early ideas, but were quickly dismissed. We implemented this by adding I2C-capable IO expanders to each button module and I2C-capable ADCs to each joystick module. By scanning for I2C addresses, we were able to detect a module's presence, as well as determine its type. The original plan was to store each module's information data on an EEPROM, but we ran into some issues regarding this. Each module has an EEPROM soldered onto it, but our firmware does not utilize them. Input data from the buttons/joysticks is read by the IO Expander or ADC and then transmitted to the MCU through I2C.

Mechanical and Ergonomic Design

The physical design of the controller is built to house the main receiver board. The consistent shape and sizing enable the modularity of the controller. Each module, Joystick, Buttons, and Bumper, all have the same footprint for both ease of manufacturing and symmetry. The 3D-printed enclosure provides sufficient internal space for all modules. Extended side grips

were added to improve handling during gameplay without compromising the modular placement of receivers.

Summary

This methodology established a repeatable framework for modular hardware development: from microcontroller selection and USB communication setup to enclosure design and fabrication. The resulting prototype demonstrates reliable USB HID communication, functional modular inputs, and a flexible architecture ready for continued expansion and refinement.

Bill of Materials:

The current design uses a single, updated controller configuration built around the Seeed Studio XIAO ESP32-S3 Plus as the main microcontroller. This approach simplifies the overall system by standardizing the hardware across all modules and reducing assembly complexity. The design is divided into several functional boards including the receiver, button, joystick, bumper, and navigation boards each with its own supporting components such as EEPROMs, I/O expanders, resistors, and capacitors. Custom PCBs are used for each subsystem, along with pogo pin connections and JST connectors to enable modularity and easier assembly. This structure allows for a flexible and scalable controller design while keeping the total cost and component count manageable.

Table 1: Bill of materials

Part name	Desc	Qty.	Total	Link
Receiver Board		Total		
LTC4317	Address Translator	4	\$20.72	Digikey.com
24LC02BT-I/OT	2 kbit EEPROM	1	\$0.31	Digikey.com
PCF8574DWR	8 bit io expander	1	\$1.99	Digikey.com
603_1208 metric	10 uF capacitor	3	\$0.03	Digikey.com
603_1208 metric	.1 uF capacitor	4	\$0.24	Digikey.com
603_1208 metric	10 kΩ resistors	27	\$0.24	Digikey.com
XIAO ESP32-S3 Plus	Seeed Studio XIAO ESP32-S3 Plus	1	\$7.90	Seeedstudio.com
PCB	Main Receiver Board	1	\$29.00	
603_1208 metric	0 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	102 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	182 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	280 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	392 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	523 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	681 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	887 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	976 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	1 MΩ resistors	5	\$0.40	Digikey.com
1x8 smd pogo pins	Pogo pin connection	8	\$42.48	Digikey.com
JST SH 4-pin Connector	JST SH 4-pin Right Angle Connector (10-pack) - Qwiic Compatible	1	\$3.95	Adafruit.com
Subtotal:		65	\$108.16	

Part name	Desc	Qty.	Total	Link
Button Board				
24LC02BT-I/OT	2 kbit EEPROM	1	\$0.31	Digikey.com
PCF8574DWR	8 bit io expander	1	\$1.99	Digikey.com
SKQGABE010	Button	4	\$1.24	Digikey.com
PCB	Button Board	1	\$7.00	
Subtotal:		7	\$10.54	
Joystick				
24LC02BT-I/OT	2 kbit EEPROM	1	\$0.31	Digikey.com
ADS1115IDGST	ADC	1	\$6.29	Digikey.com
603_1208 metric	10 kΩ resistors	1	\$0.10	Digikey.com
9032	Joysticks	1	\$5.25	Digikey.com
PCB	Joysticks Board	1	\$7.00	
Subtotal:		5	\$18.95	
Bumper Board				
24LC02BT-I/OT	2 kbit EEPROM	1	\$0.31	Digikey.com
PCF8574DWR	8 bit io expander	1	\$1.99	Digikey.com
PCB	Bumper Board	1	\$7.00	
Key switches	Switches for bumpers and triggers	1	\$4.49	Amazon.com
Subtotal:		4	\$13.79	
Nav/master Board				
24LC02BT-I/OT	2 kbit EEPROM	1	\$0.31	Digikey.com
PCF8574DWR	8 bit io expander	1	\$1.99	Digikey.com
LTST-C193TBKT-5A	Blue LED	4	\$0.72	Digikey.com
B3F-1000	Button	4	\$0.96	Digikey.com
PCB	Nav Board	1	\$7.00	
319-10-104-00-006000	pogo pin strikers	3	\$8.94	Digikey.com
Subtotal:		14	\$19.92	
Total:		95	\$171.36	

Tools Used:

Software

Table 2: Software used

Name	Description	Price	Link
GitHub Desktop	Used to share updates and versions.	Free	GitHub.com
Arduino IDE	Used as the programming platform.	Free	Arduino.cc
KiCAD	The program was used for creating the PCB.	Free	KiCAD.com
OnShape	Used for the frame design.	Free	OnShape.com
Ultimaker Cura	Used as a slicer for the 3-D printed glasses frame.	Free	Ultimaker.com
Total Price			

Hardware

Table 3: Hardware used

Name	Description	Price	Link
3D-printer	Ender-3 printer with PLA 1.75mm used for the frames of the glasses.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	3D-Printer
Name	Description	Price	Link
Solder Iron	Used for touch-ups and adding components.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Solder Gun
Heat gun	The heat gun was used to help get the ESP32 on the boards and align some of the components.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Heat Gun
Multimeter	The Multimeter was from the lab and was used to test the PCB.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Multimeter
Sandpaper	The sandpaper was used to clean up the edges of the PCB. A finer grit would be ideal for this situation.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Sandpaper
Tweezers	The tweezers were used to hold components in place when soldering.		
Total Price		Varies	

Assembly

1. Gather Materials

- a. Refer to the BOM
- b. Download the main GitHub repo branch ([link](#))

2. Order PCBs

- a. Use our PCB manufacturer; the Gerber files are provided in the GitHub repository.

3. Print the Shells

- a. The shells for the modules are located in the Github directory under Hardware/Module STLs
- b. The standard configuration uses seven modules and each module requires the specified parts
 - i. 1 Master: 1 Master top casing, 1 Master bottom casing, 3 Master buttons
 - ii. 2 Bumpers: 1 Joystick top casing, 1 Joystick bottom casing
 - iii. 2 Joysticks: 1 Bumper body casing and 2 Bumper caps
 - iv. 2 Buttons: 1 Button top casing, 1 Button Bottom casing, 4 non-Master buttons
- c. The shell for the main body is located in the Github directory under Hardware/Shell/OSMC
 - i. The full casing can be printed from the .step file
 - ii. Individual parts can be printed from the .stl files

4. Prep the PCB

- a. Lightly sand away any edge defects from manufacturing making sure not to
- b. Lightly wipe with isopropyl alcohol

5. Solder Components

– In this production, blank boards were ordered, and most components were hand-soldered with a soldering iron, solder paste, and hot air soldering station.

a. Receiver Board

- i. Start by soldering on all the SMD components such as the capacitors, resistors, and address translators. This is done easiest using a reflow oven or a hot plate.
- ii. Next, solder all of the pogo pins/pin headers and through hole components.
- iii. Finally, solder on the JST connectors if you want to have any off board modules.

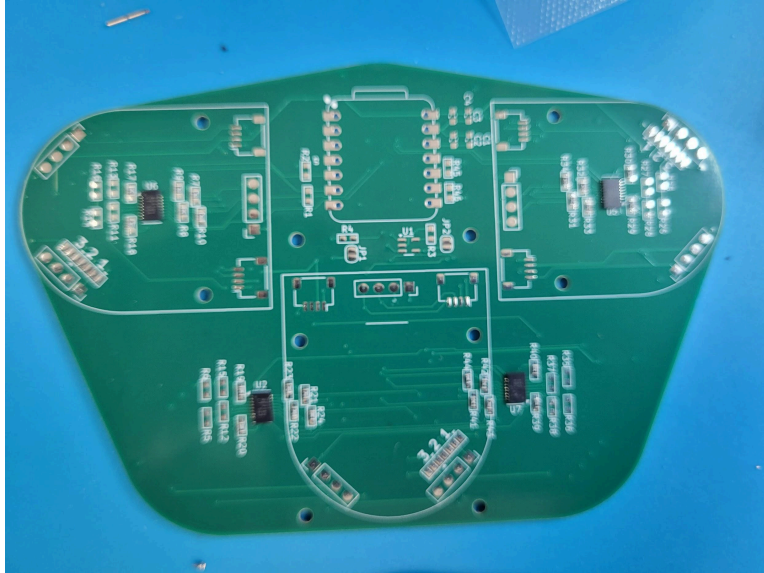


Figure 4: Receiver Board Module

b. Joystick Modules

- i. Solder the joystick into the designated positions on the PCB, keeping each button flat against the board.
- ii. Inspect all joints to confirm solid solder connections and verify that the buttons click freely without obstruction.
- iii. Place the module in the joystick module shell, and mark the shell with where the up arrow is pointing.

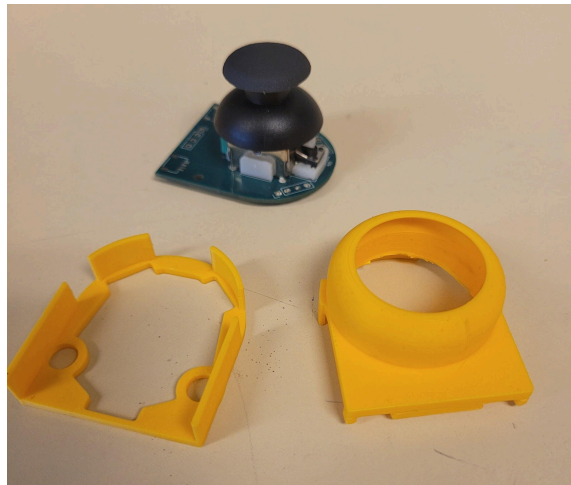


Figure 6: Joystick Module (soldered, apart)

c. Button Modules

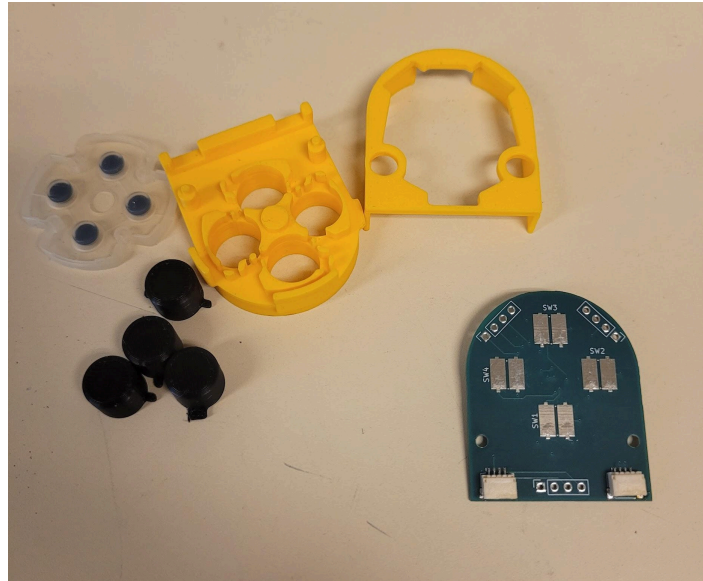


Figure 7: Button Module (membrane, apart)

- The button modules can be assembled in two different ways. This depends on if you want to use traditional membrane buttons or solder on mechanical buttons.
- i. If you would like to use mechanical buttons, solder the four push buttons into their designated labeled positions on the PCB, keeping each button flat against the board.

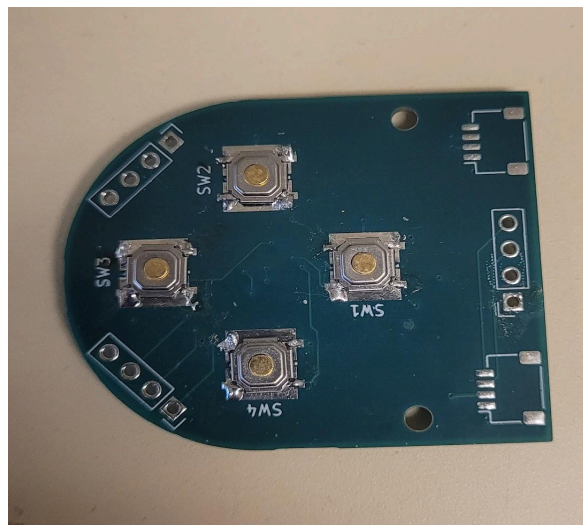


Figure 8: Button Module (mechanical, soldered)

- ii. Solder EEPROM and IO expanders to the back of the board. This is done easiest with a reflow oven or a hot plate.
- iii. If you don't use conventional connector pins, you can solder in the JST connectors at this point.

- iv. (Optional) Glue the top plate onto the module.

d. Master/Navigation module

- i. Solder EEPROM and IO expanders to the back of the board. This is done easiest with a reflow oven or a hot plate.
- ii. Solder the mechanical buttons onto the board.
- iii. Solder on the spacer pogo striker headers.

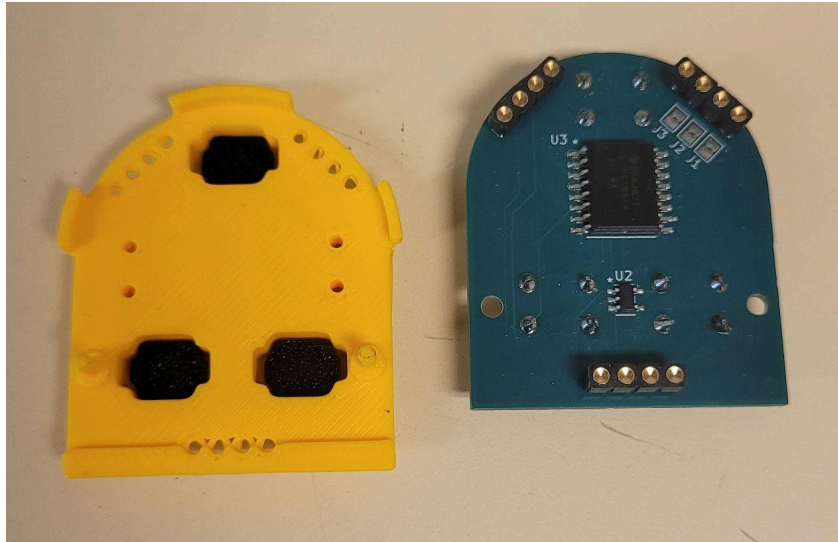


Figure 9: Master/Navigation Module (soldered, apart)

- iv. (Optional) Glue the top plate onto the module.

e. Bumper/Trigger module

- i. Solder EEPROM and IO expanders to the back of the board. This is done easiest with a reflow oven or a hot plate.
- ii. Solder the through hole key switches onto the board.

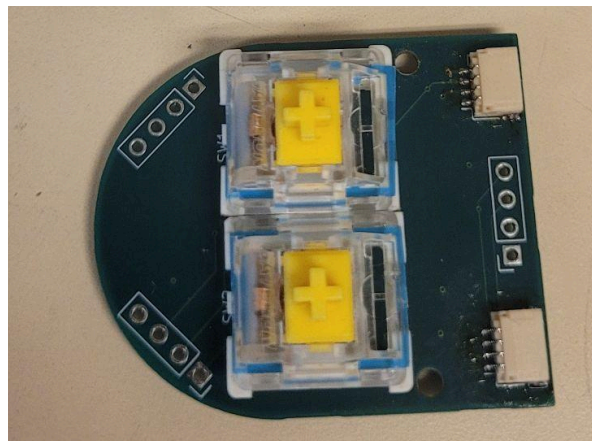


Figure 10: Bumper/Trigger Module(soldered)

- iii. Solder the JST connectors
- iv. Optional) Glue the top plate onto the module.

6. Uploading code

- a. Install Arduino IDE.
- b. Add ESP32 boards manager URL:
 - i. In Arduino IDE, select File -> Preferences. Paste this URL in the additional boards manager URL box:
"https://espressif.github.io/arduino-esp32/package_esp32_index.json".
- c. Install the ESP32 board package.
 - i. Next, select "Tools" -> "Board: (whatever)" -> "Boards Manager". Search for ESP32 and install the board package from Espressif Systems.
- d. Install the Adafruit ADS1115 library
 - i. Next, select "Tools" -> "Manage Libraries...". Search Adafruit_ADS11X5 and install the library by Adafruit .
- e. Import project files into Arduino IDE.
- f. Connect via USB.



Figure 11: Seeed Studio XIAO ESP32-S3 Plus

- g. Select the **ESP32-S3 Dev Module** and the correct COM port.
 - i. The COM port could be different depending on your computer.

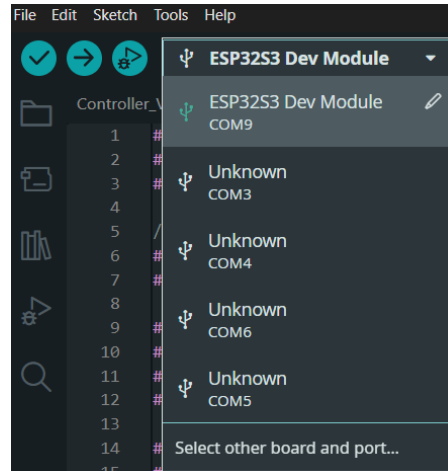


Figure 12: Arduino IDE COM Selection

- h. Select “Tools”, then change to the match following settings.

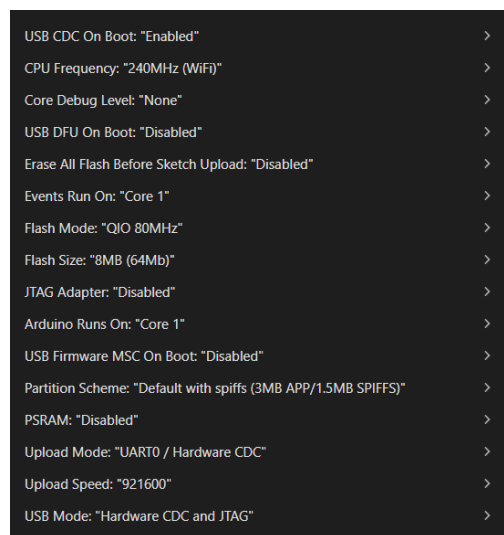


Figure 13: Board Configuration

- i. Compile and upload the sketch onto the ESP32 via USB-C.

7. Assemble in-shell

- a. Place receiver board in shell and mark screw hole locations
- b. Remove the receiver board and embed the M3 heat set inserts where you just marked, making sure the inserts are level with the surface.
- c. Repeat steps a and b for the two bumper/trigger modules
- d. Wire the JST connectors from the bumper/trigger modules, through the slots, connecting to the JST connectors on the backside of the receiver board

- e. Screw the receiver board and bumper/trigger modules in their respective slots with M3 screws through the PCBs and into the placed heat set inserts
- f. Place the master module resting on the center module pogo pins.
- g. Place the joystick/button modules, in the configuration chosen, resting on the outer pogo pin sets.
- h. Push the top plate into place, making sure all inputs align with their respective holes.
- i. Push the bottom plate into the main shell
- j. To move top modules, push up on the bottom tab of the top plate, and repeat steps g. - i. with the new configuration

8. Testing

- a. Use a program such as **Steam Input** or **gamepad-tester**.
- b. Verify that every button and joystick direction registers properly.

Characterization Data:

Input Testing

Testing the inputs was straightforward. First was testing all the buttons and joysticks. This was tested within the gamepad tester website. The website shows a picture of a controller on the website and highlights the input that it thinks it is receiving from our controller. From this test, we found that the top left module on the controller face doesn't work; this is explained more in the hardware continuity section.

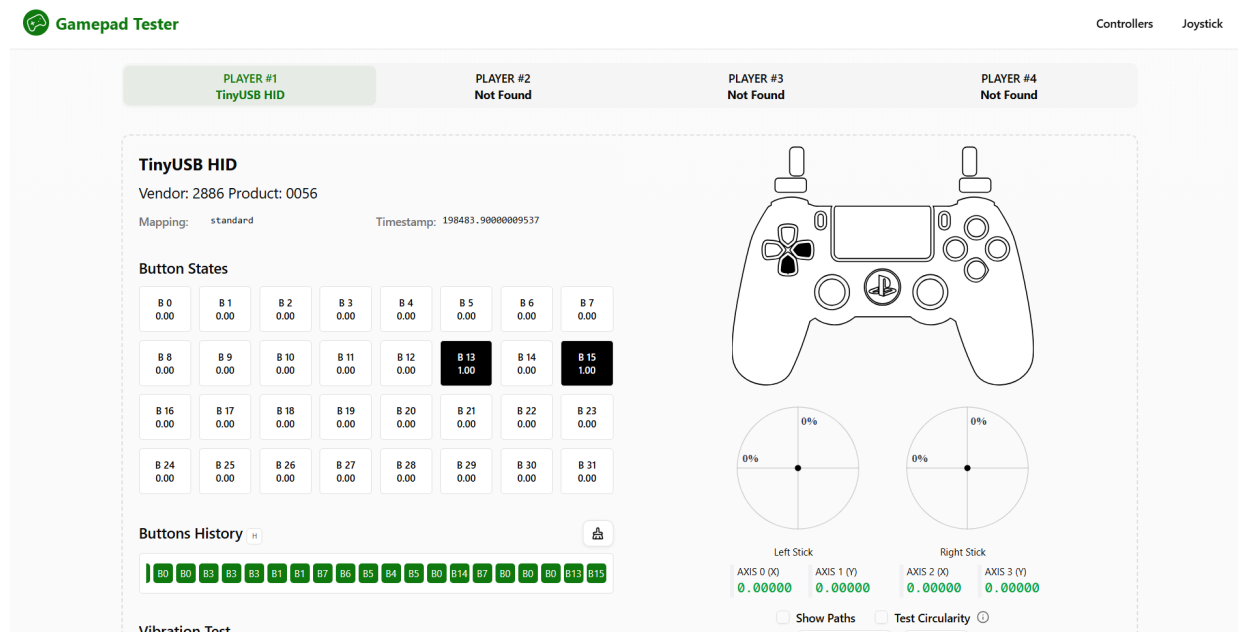


Figure 13: Gamepad Tester Example

Ergonomic Design

To see how the controller's ergonomics felt to others, we asked random people at Michigan Tech to hold the controller and write down their thoughts. The responses were mostly positive, with the main comments being that the size was too big, specifically relating to the triggers.

Table 4: Ergonomics Feedback

Name of Reviewer	Comments
Matt	I like the modularity
Kyle	Cool idea!
Daniel	Fits my hand great, triggers are a bit big.
Kegan	Works fine, dislike the back and how my fingers fit on it
Gabriel	Joysticks are far from buttons
Brandan	Great fit!
Madison	A little big for smaller hands
Austin	No place to rest my fingers, would be accidentally pressing triggers.
Jonathan	Bolstering by middle + ring finger is a bit thin
Patrick	Top controllers too far.

Hardware Continuity

When designing the board, there was a misplace on the i2c lines on the left module slots. This stopped the communication with the two modules on that side that prevented the addressing of the modules. To fix this problem wires were routed from the i2c lines off the microcontroller to reach over to that side to the i2c lines and the address translator of that side was removed. In the current pcb revision, this has been fixed by reducing the number of components that the signal passes by to get to the address translator.

The other problem was that the data and clock line of the module upper left module were swapped. This caused the module to not be detected and go down the rabbit hole of why that module does not work. This module slot was not able to get working but was fixed on the pcb revision.

Standards:

Open Source Hardware Standards [1]:

1. Documentation
2. Scope
3. Necessary Software
4. Derived Works
5. Free Redistribution
6. Attribution
7. No Discrimination Against Persons or Groups
8. No Discrimination Against Fields of Endeavor
9. Distribution of License
10. License Must Not Be Specific to a Product
11. License Must Not Restrict Other Hardware or Software
12. License Must Be Technology-Neutral

Recommendations for Future Work:

Some recommendations for future additions to this project should focus on improving both manufacturability and functionality of the system. One key area is the use of more hand-solderable components, which would simplify assembly, reduce reliance on specialized equipment, and make iterative prototyping more accessible. Changing the IC on the joystick modules to use a larger chip would allow more addresses without the use of the address translators and be more hand solderable. Expanding the code to enable more advanced input mapping, allowing the device to support customizable controls and more complex interaction schemes without external computation. Further PCB refinement and testing will be important to enhance reliability, signal integrity, and overall performance. Integrating a wireless mode using Bluetooth HID would significantly increase usability and portability by removing the need for physical connections. Finally, exploring additional module types—such as battery integration, gyroscopes, sliders, and touchpads—would broaden the system's versatility and open up new application possibilities.

Sponsors/Acknowledgements:

This project was self-sponsored by the Open Source Hardware Enterprise at Michigan Technological University.

References:

- [1] "OSHW - Definition of Free Cultural Works," Freedomdefined.org, 2023.
<https://freedomdefined.org/OSHW>
- [2] "Alpakka Controller," Input Labs, <https://inputlabs.io/alpakka> (accessed Dec. 5, 2025).
- [3] "ESWAP x 2 pro controller," ESWAP X2 PRO CONTROLLER | Thrustmaster USA,
https://eshop.thrustmaster.com/en_us/eswap-x2-pro-controller.html (accessed Dec. 5, 2025).
- [4] "Gamepad Tester," Hardware Tester, <https://hardwaretester.com/gamepad> (accessed Dec. 5, 2025).
- [5] "HID Usage Tables FOR Universal Serial Bus (USB)." Available:
https://usb.org/sites/default/files/hut1_6.pdf (accessed Sep. 20 2025)
- [6] "Proteus Controller™ kit," ByoWave Proteus Controller,
<https://byowave.com/products/proteus-controller> (accessed Dec. 5, 2025).
- [7] "TinyUSB Library," GitHub, <https://github.com/chegewara/EspTinyUSB> (accessed Dec. 5, 2025).

Appendix:

Michigan Technological University

Enterprise Program

Project Report Checklist

CHECKLIST FOR PROJECTS ENT 4950/60

A copy of this checklist must be submitted with the project report.

ABET Requirement:

1. Have you reviewed relevant industry standard(s) pertaining to your project and applied the necessary ones?	Yes ☒	
2. Have you included a list of the standards used in project documentation? Where: Standards section	Yes ☒	
3. Has your design considered public health, safety, and welfare	Yes ☒	Not Applicable ☐
4. Has your design/solution considered the following factors:		
a. global factors	Yes ☒	Not Applicable ☐
b. cultural factors	Yes ☒	Not Applicable ☐
c. social factors	Yes ☒	Not Applicable ☐
d. environmental factors	Yes ☒	Not Applicable ☐
e. economic factors	Yes ☒	Not Applicable ☐

Project work

1. Does the report have a one page Executive Summary that can be read on its own?	Yes ☒	No ☐
2. Does your project report have a clear problem statement	Yes ☒	No ☐
3. If this is a continuation from a previous semester/ year, is the previous work described appropriately in the report?	Yes ☒	No ☐
4. Have you described the relevant constraints to your design/solution?	Yes ☒	No ☐
5. Have you documented the different design options considered before arriving at the final design/solution?	Yes ☒	No ☐
6. Have you included recommendations for future work	Yes ☒	No ☐

Project Sponsor

1. Have you received explicit permission from your sponsor to share details about the project or client?	Yes ☒	No ☐
--	---	-----------------------------